人工智能基础及应用(通识课 C)实验指导书

实验 1 语义网络写入图数据库

1.1 实验目的

1. 了解语义网络的基础内容及其应用场景；
2. 掌握语义网络表示知识的方法；
3. 掌握 gStore 呈现语义网络的使用方法；
4. 培养学生自主学习、探索和实践的能力。
5. 通过小组讨论和创意应用环节，培养学生的团队协作能力和创新思维。

1.2 实验内容

1. 注册 gStore Cloud 账号

http://portal.gstore.cn/login.html?pcode=g004

点击【立即注册】

2. 创建一个图数据库

登录 gStore Cloud 网站，进入【会员专区】--【数据库管理】，点击【启动实例】，点击【新建】，弹出对话框，选择构建方式为【示例数据库】，示例数据库为【空白数据库】。创建过程如图 4-10 所示。



图 4-10 创建图数据库过程

3. 将事实转化为语义网络（可以在纸上画出来）
4. 根据语义网络编写 RDF 数据
5. 将 RDF 数据封装成 SPARQL 语句(非常简单）(注意：每一条 RDF 数据用.隔开）

Insert data { RDF 数据 }，如，RDF 数据为 <感冒灵颗粒> <适用> <感冒> . 以及<感冒灵颗粒> <名称> “感冒灵颗粒”，则生成的 SPARQL 语句为

Insert data {<感冒灵颗粒> <适用> <感冒> . <感冒灵颗粒> <名称> “感冒灵颗粒”.}

6. 编写 Python 程序

导入 gStore api 包：在 terminal 窗口输入：pip3 install gstore-api

编写函数依次完成如下功能

获取数据库实例的 KEYID 和 Secrete

在 gStore Cloud 的主页里面可以查看

7.调用 gStore api 的函数完成语义网络入库操作

queryDB 函数（具体参考 demo.py 文件）

8.在 gStore Cloud 里面可视化语义网络

【会员专区】-【数据库查询】，选择自己创建的数据库，输入以下 SPARQL 语句

Select * where {?x ?y ?z.}

【示例程序】

Demo.py

```
# 导入 python 包
from gstore_api import gStoreCloud

# 创建 api 对象
## keyid 为 gStoreCloud 主页里面的 KeyID
## secret 为 gStoreCloud 主页里面的 Secret
gstoreapi=gStoreCloud.gStoreCloud(keyid,secret)

# 插入数据到图数据库中
## dbname 为数据库名称
## sparql 为插入的 sparql 语句
result=gstoreapi.queryDB(dbname,sparql)
```

Sparql 语句示例（示例里面有 4 条 RDF 数据，每条数据用点号隔开）

```
sparql="insert data {<张三> <姓名> \"张三\". <张三> <年龄> \"23\". <李四> <年龄> \"40\".
<张三> <朋友> <李四>.}";
```

1.3 实验任务

1.借助大语言模型工具编写程序代码，并将程序代码的每一行都加上注释，将添加了注释的程序代码粘贴到 Word 文档。

2.运行程序，将程序的运行结果截图粘贴到 Word 文档。

3.分析程序的运行结果，将分析过程写到 Word 文档。

4.实验报告以 Word 文档的形式提交到畅课系统中。

实验 2 数据可视化与决策分类

2.1 实验目的

1. 了解数据可视化相关知识
2. 掌握基于 matplotlib 的直方图、条形图等可视化实现。
3. 掌握基础的决策树分类以及决策可视化。

2.2 实验内容

1. 直方图及实现

直方图（Histogram）是表示数据分布的一种图，它通过把数据分成一组连续的、不重叠的区间（即“桶”或“bins”）来显示每个区间的数据点数，这些数据点以柱状图的方式绘制。

可以使用 matplotlib 画出直方图，见图 4-11。具体代码如下，其余代码托管于链接：

```
x='Age'
n, bins, patches = plt.hist(df['Age'], bins="auto", color='skyblue', edgecolor='black', linewidth=1)
for i in range(len(bins) - 1):
    bin_start = bins[i]
    bin_end = bins[i + 1]
    bin_label = f"[{bin_start:.1f}, {bin_end:.1f})"
    bin_height = n[i]
    if bin_height == 0:
        continue
    plt.text((bin_start + bin_end) / 2, -0.5, bin_label, ha='center', va='top', fontsize=10, rotation=90)
    plt.text((bin_start + bin_end) / 2, bin_height, int(bin_height), ha='center', va='bottom', fontsize=10)
plt.xticks([])
```

matplotlib.pyplot.hist 的参数如下：

- bin 指定直方图的区间（桶）数量或边界。
- range 指定数据的范围
- density 是否将直方图标准化为概率密度函数（即所有柱子的面积和为 1）。
- weight 为每个数据点分配权重，影响每个柱子的高度。
- color 指定直方图的颜色。
- edgecolor 指定直方图边缘的颜色。
- linewidth 指定直方图边缘的线的宽度。

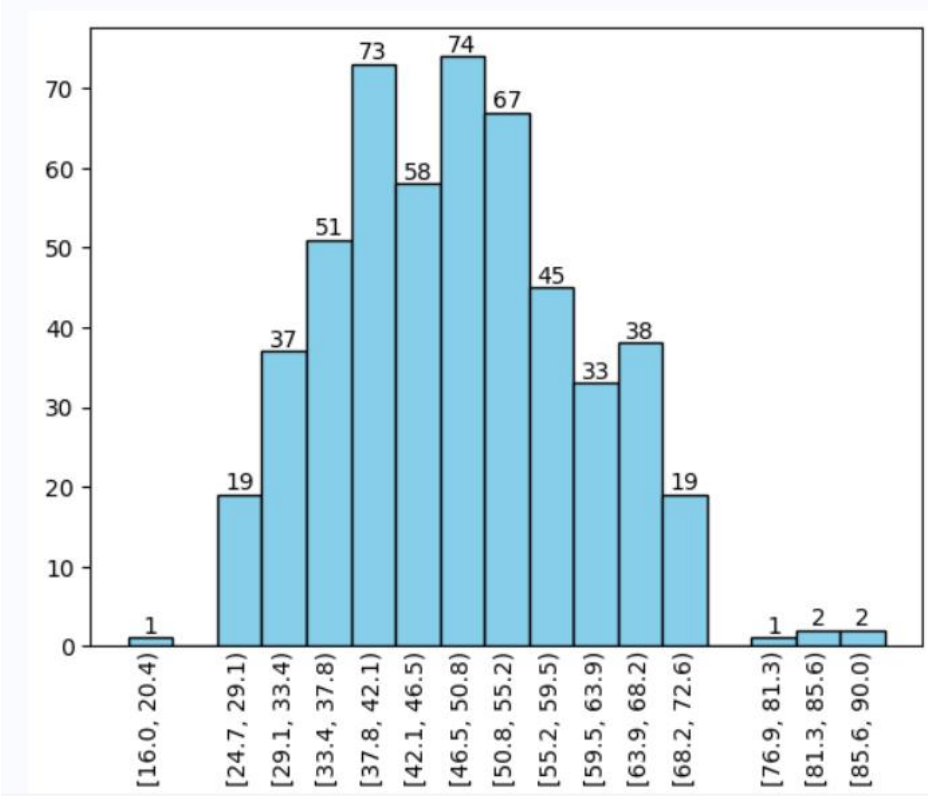


图 4-11 直方图

可见该数据集的年龄跨度大，数据代表性强、多样性高以及适用性广。

2. 条形图及实现

条形图（Bar Chart）是一种表示分类数据结果可视化方法，用水平或垂直条形来表示每个不同类别数值大小。条形图通常用于比较不同类别之间数量、频率或占比关系，是数据分析和可视化中常用图表之一，如图 4-12 所示。

matplotlib.pyplot.bar 常用参数：

- x：浮点数或类似数组，表示条形 x 坐标，即类别标签位置。
- height：表示条形高度，即每个类别数值大小。

w idth：默认为 0.8，即条形宽度。b el l：浮点数或数组，默认为 0，即条形底边上。y 坐标。设置该参量可以使堆叠条形图效果。

- center:可选择“center”或“edge”，值为“center”时，为 x 轴对齐条形。为“edge”，为左边缘对齐 x 值。
- color：用于指定条形颜色。
- edgecolor：用于指定条形边缘颜色。
- linewidth：用于指定条形边缘宽度。

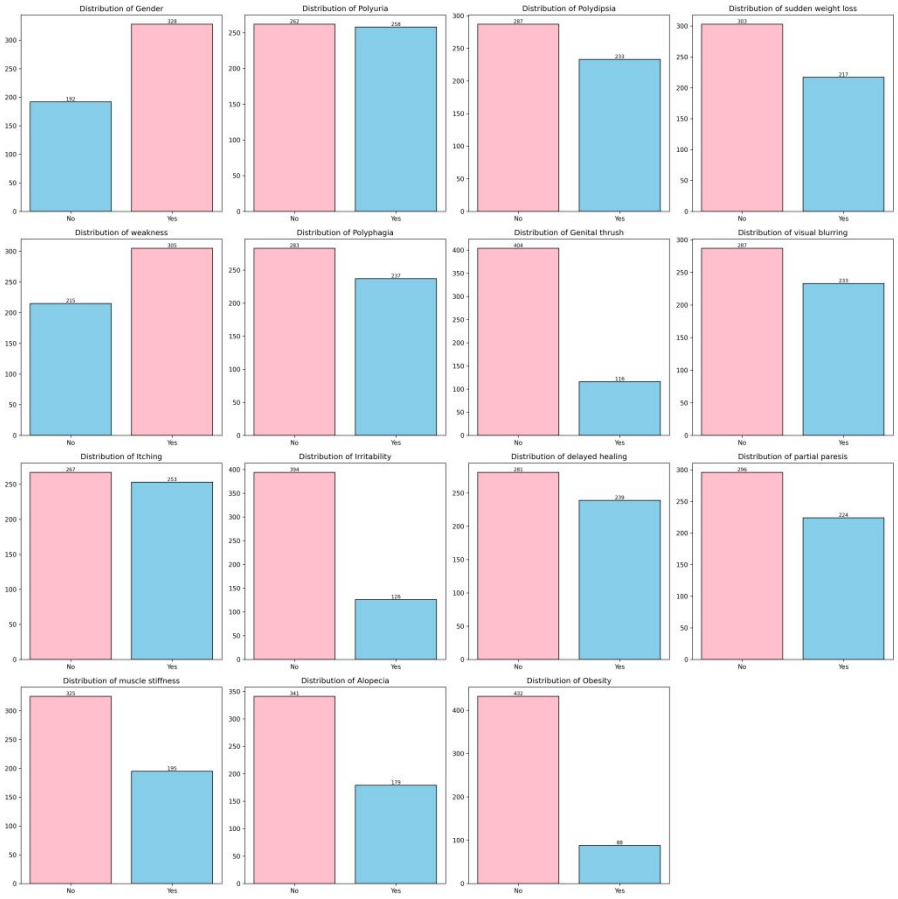


图 4-12 每一特征对应的条形图

3. 相关性矩阵及实现

相关系数（RelatedCorrelation）是所有特征两两之间数量关系。相关性矩阵（CorrelationMatrix）是表示多个变量之间线性相关系数矩阵。其主对角线全为 1（任何变量与它自身相关系数为 1），并且具有对称性。1 附近表示强正相关（一个变量越大，另一个变量越大），-1 附近表示强负相关（一个变量越大，另一个变量越小），0 附近则表示几乎没有线性关系。

使用 Python 的 DataFrame.corr()方法能够便捷地计算出相关性矩阵，默认计算是皮尔逊相关系数（Pearson CorrelationCoefficient）。图 4-13 是特征间相关性矩阵。

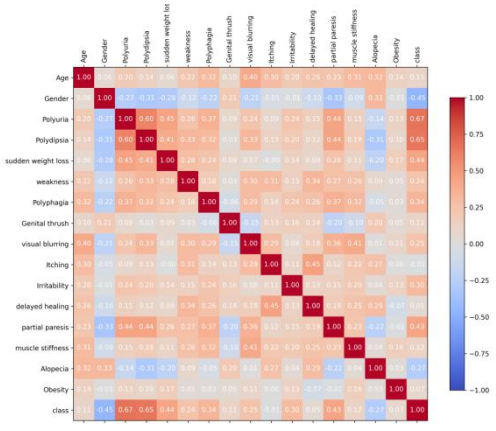


图 4-13 相关性矩阵

4. 决策树分类与可视化介绍以及实现

决策树分类，作为一棵基于决策规则构建树状模型，是监督学习领域一项核心算法。它专注于解决各类分类任务，其运作原理是通过不断细化数据集，从而实现对于数据中不同类别有效区分。该模型优势在于其高度直观性和可解释性，这使得决策树分类模型不仅在理论研究中受到重视，而且在实际应用领域，例如在医疗诊断、金融风险评估以及客户细分等方面均显示出了其卓越价值和实用性。

决策树通过不断地选择最佳特征并且根据特征值将数据集分割为子集，直到满足某个停止准则前（最大深度、节点纯度足够高或者数据集无法再划分），每次划分目标都是使得子集纯度尽可能高（即每个特定子集中尽可能包含属于同一个类别样本）。

下列是决策树分类以及可视化代码：

```
# 准备数据
X = df.drop(columns=['class'])  # 特征列
y = df['class']  # 目标列
# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 初始化并训练决策树模型
tree_model = DecisionTreeClassifier(max_depth=4, random_state=42)  # 限制树的深度以便可视化
tree_model.fit(X_train, y_train)
# 预测
y_pred = tree_model.predict(X_test)
# 评估模型
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
# 可视化决策树
plt.figure(figsize=(20, 10))
plot_tree(tree_model, feature_names=X.columns, class_names=['Negative', 'Positive'],
          filled=True, rounded=True, max_depth=4)
plt.title("Decision Tree Visualization")
plt.show()
```

DecisionTreeClassifier 是 Scikit-Learn 中用于构建决策树分类器的类

- max_depth 决策树的最大深度。
- random_state 随机种子，用于控制随机性.
- criterion 用于选择特征的分裂标准。默认为基尼不纯度
- min_samples_split 分裂内部节点所需的最小样本数。

最后，其决策树可视化结果如下图 4-14。

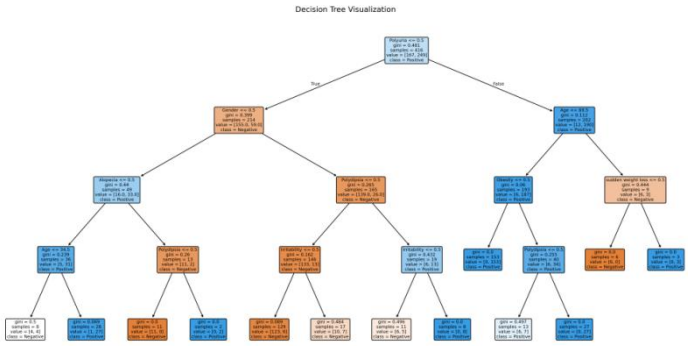


图 4-14 决策树

最后，在决策树可视化中，特征重要性可以通过其在树中位置和使用频率来判断分裂深度：越靠近树根特征（即在树的上层被用于分裂的特征）通常越重要，因为它们对数据的划分起到了关键作用。分裂次数：一个特征在树中被用于分裂的次数越多，通常说明其对模型的贡献越大。故由上图可知，多尿（Polyuria）、多饮（Polydipsia）、易怒（Irritability）在早期糖尿病的判断中起着较大的作用。

2.3 实验任务

1.借助大语言模型工具编写程序代码，并将程序代码的每一行都加上注释，将添加了注释的程序代码粘贴到 Word 文档。

2.运行程序，将程序的运行结果截图粘贴到 Word 文档。

3.分析程序的运行结果，将分析过程写到 Word 文档。

4.实验报告以 Word 文档的形式提交到畅课系统中。

实验 3　Transformer 语音识别

3.1 实验目的

1. 用 PaddleSpeech 加载预训练的 Transformer 语音识别模型。
2. 对输入的音频文件进行语音识别，输出对应的文本。
3. 对自定义数据集进行微调，以适应特定场景。
4. 通过小组讨论和创意应用环节，培养学生的团队协作能力和创新思维。

3.2 实验内容

1. 安装并更新 paddlepaddle 工具包

使用 pip 安装 paddlepaddle 工具包：

```
pip install paddlepaddle
```

2. 安装 paddlespeech 工具包及其相关依赖

PaddleSpeech 是基于 PaddlePaddle 的语音方向的开源模型库，用于语音和音频中的各种关键任务的开发，包含大量基于深度学习前沿和有影响力的模型，一些典型的应用如：声音分类、语音识别、语音翻译、语音合成等。

相关依赖如下：

gcc >= 4.8.5

paddlepaddle >= 2.3.1

python >= 3.7

linux(推荐), mac, windows

win 必须安装 Microsoft C++生成工具

3. 在 Windows 系统上安装 GCC 编译器

安装步骤：

下载安装包：访问 MinGW 官方网站 下载 MinGW 安装管理器（MinGW Installation Manager）。

运行安装程序：打开下载好的安装程序，按照提示完成基本的安装设置。

选择组件：在安装管理器中，选择要安装的组件，至少要勾选 mingw32-gcc-g++（用于编译 C 和 C++ 代码）和 mingw32-base。

执行安装：选择 Installation -> Apply Changes 开始安装所选的组件。

配置环境变量：安装完成后，需要将 MinGW 的 bin 目录（通常是 C:\MinGW\bin）添加到系统的环境变量 PATH 中。具体操作是：右键点击 “此电脑” -> “属性” -> “高级系统设置” -> “环境变量”，在系统变量中找到 PATH 变量，编辑并添加 MinGW 的 bin 目录。

验证安装：打开命令提示符（CMD），输入 gcc --version，如果能正确显示 GCC 的版本信息，则说明安装成功。

4. windows 安装 Microsoft C++生成工具

下载 Visual Studio Build Tools 安装程序：访问 Visual Studio Build Tools 下载页面。点击 “下载 Visual Studio Build Tools” 按钮，下载安装程序。

运行安装程序：下载完成后，运行安装程序，在 “工作负载” 选项卡中，如果你只是需要基本的 C++ 开发工具，可以选择 “使用 C++ 的桌面开发” 工作负载，它包含了 Microsoft C++ 生成工具以及其他常用的开发组件。若你希望更精细地选择组件，可切换到 “单个组件” 选项卡，在 “编译器、生成工具和运行时” 类别下，勾选 “MSVC vXXX - VS 20XX C++ x64/x86 生成工具”（XXX 和 XX 代表具体版本号，根据你的需求选择合适的版本），同时也可按需勾选其他相关组件，如 Windows SDK 等。

完成安装：选择好组件后，点击 “安装” 按钮，等待安装完成。安装完成后，你可以通

过以下步骤验证 Microsoft C++ 生成工具是否安装成功：

打开“命令提示符”或“PowerShell”。

输入 cl.exe 命令并按回车键。如果安装成功，会显示编译器的版本信息和使用说明；若提示“‘cl.exe’不是内部或外部命令，也不是可运行的程序或批处理文件”，则说明安装可能存在问题，需要检查安装步骤或重新安装。

5. 安装 paddlespeech 工具包

使用 pip 安装 paddlepaddle 工具包：

pip install paddlespeech -i https://pypi. tuna. tsinghua. edu.cn/simple/

6. 基于 transformer 模型的语音识别代码

基于 paddlespeech 模型的语音识别代码如下：

```
# 导入必要的库
import paddle
from paddlespeech.cli.asr.infer import ASRExecutor

# 设置使用的模型，这里选择一个常见的模型
# 'transformer' 是 ASR 模型之一
# 可以选择其它模型，比如'conformer', 'wav2vec2', etc.
model = "transformer"

# 初始化 ASRExecutor
asr = ASRExecutor()

# 输入音频文件的路径
audio_file = "ie.wav"

# 语音转文本
result = asr(audio_file=audio_file, force_yes=True)  # 支持 16k，其它采样率强制转换
# 输出转录结果
print("识别结果：", result)
```

多数情况下，PaddleSpeech 推荐使用 16kHz 的采样率。这是因为许多预训练模型是基于 16kHz 采样率的音频数据进行训练的，使用该采样率能让模型发挥出较好的性能。不过，部分功能支持自动将其他采样率的音频转换为 16kHz，比如在语音识别时设置 force_yes=True 可对非 16k 采样率音频进行强制转换。

7. 代码运行结果

运行上述代码，结果显示如下：

```
识别结果：早上八点我从北京到广州花了四百二十六元
```

3.3 实验任务

1.借助大语言模型工具编写程序代码，并将程序代码的每一行都加上注释，将添加了注释的程序代码粘贴到 Word 文档。

2.运行程序，将程序的运行结果截图粘贴到 Word 文档。

3.分析程序的运行结果，将分析过程写到 Word 文档。

4.实验报告以 Word 文档的形式提交到畅课系统中。

实验 4　Stable Diffusion 图像生成

4.1 实验目的

1. 了解百度飞桨的 PaddleHub 深度学习工具包、Stable Diffusion 模型的基本原理。
2. 掌握基于 PaddleHub 的 Stable Diffusion 图像生成方法并实现。
3. 培养学生自主学习、探索和实践的能力。
4. 通过小组讨论和创意应用环节，培养学生的团队协作能力和创新思维。

4.2 实验内容

1. 安装并更新 PaddleHub 工具包

使用 pip 安装并更新 PaddleHub 工具包：

```
pip install --upgrade paddlehub
```

2. Stable Diffusion 模型接口说明

使用 module = hub.Module(name='stable_diffusion') 代码即可调用 PaddleHub 工具包中的 Stable Diffusion 模型，该模型的接口定义如下：

```
def generate_image(
text_prompts,
style: Optional[str] = None,
artist: Optional[str] = None,
width_height: Optional[List[int]] = [512, 512],
seed: Optional[int] = None,
batch_size: Optional[int] = 1,
output_dir: Optional[str] = 'stable_diffusion_out'):
```

其中，参数 text_prompts(str)为输入描述想要生成图像的文本内容，通过构造方式为“一段描述性文字内容”+“指定艺术家的名字”，具体构造方式可参照其说明文档；参数 style 为指定绘画风格，如'watercolor'、'Chinese painting'等；参数 width_height 为输出生成图像的宽高，宽高需是 64 倍数，默认为[512, 512]；参数 seed 为随机种子；参数 batch_size 为每个 prompt 一次生成图像的数量；参数 output_dir 为保存输出图像的目录，默认目录为'stable_diffusion_out'。该模型接口返回 DocumentArray 对象，包含 n_batches 个保存迭代过程所有中间结果的 Documents。

3. 基于 Stable Diffusion 模型的图像生成代码

基于 Stable Diffusion 模型的图像生成代码如下：

```
# 导入 paddle 和 os 模块
import paddle
from PIL import Image
import os
# 设置模型保存的新路径
new_path = 'D:\pythonProject\DLproject\StableDiffusion\Modules'
os.environ['HUB_HOME'] = new_path
# 导入 PaddleHub 模块
import paddlehub as hub

# 如果有 GPU，设置 GPU 设备，如果没有则忽略
paddle.device.set_device('cpu')
# 设置 GPU 内存使用比例
paddle.set_flags({'FLAGS_fraction_of_gpu_memory_to_use': 0.5})
```

```
# 导入 Stable Diffusion 模型
module = hub.Module(name='stable_diffusion')
# 用户输入获取描述想要生成图像的文本 text_prompts
text_prompts=input("Please enter the content of the picture you want to generate： ")
# 生成图像
result = module.generate_image(
      text_prompts=text_prompts,
output_dir='stable_diffusion_out1')

# 将生成过程存成 gif
result[0].chunks[-1].chunks.save_gif('beautiful_painting.gif')
```

注意 new_path 的路径建议不要设置在 C 盘，模型解压后约 4G 存储空间，最好预留 15-20G 的存储空间进行解压；电脑若有 4G 以上的 GPU 显存，建议用 GPU 运算，加快计算速度和提升图片生成效果；text_prompts 的文本内容建议是英文，中文效果会差些。

4. 代码运行结果

运行上述代码，并输入如下描述图片内容的文本：

```
Chinese ancient building, lotus blooming, Waterlily pond, Magnificent and fantastic John Howe landscape, summer, clouds, Fairy tale, light effect, super wide angle, Greg Rutkowski, unreal engine, James Gurney, artstation
```

根据该文本，可以生成在夏天，荷花、睡莲盛开的池塘和古代建筑风景，且是约翰·霍夫画家水彩风格的图片，如图 4-15 所示。



图 4-15　基于 Stable Diffusion 模型的图像生成示例

4.3 实验任务

1.借助大语言模型工具编写程序代码，并将程序代码的每一行都加上注释，将添加了注释的程序代码粘贴到 Word 文档。

2.运行程序，将程序的运行结果截图粘贴到 Word 文档。

3.分析程序的运行结果，将分析过程写到 Word 文档。

4.实验报告以 Word 文档的形式提交到畅课系统中。